Московский авиационный институт

(национальный исследовательский университет)

Институт №8 «Компьютерные науки и прикладная математика»

**Лабораторная работа №1**
**по курсу**

**Питон**

# Приём работ и расширяемые анализаторы

Выполнил: Эсмедляев Федор Романович
Группа: М8О-105СВ-26
Преподаватель: Ахмед Самир

Москва, 2026

## Условие

Цель работы. Освоить устройство пакета, типизацию, модель данных, исключения, `Protocol`, композицию и тестирование контрактов на примере основы системы онлайн-школы, которая принимает сдачи лабораторных, валидирует их и запускает заменяемые анализаторы.

Задание. Работа состоит из двух этапов.

– Этап А (приём и валидация): пакет в src-layout, модель `Submission`, разбор JSON, валидация обязательных полей, часового пояса и AI-декларации; ошибка содержит `code`, `field` и `message`; различаются ошибка формата, нарушение схемы и конфликт идентификатора. Повторный `submission_id` с тем же хешем возвращает уже принятую сдачу, с другим содержимым — `conflict`. Репозиторий хранится в памяти, принятый код не исполняется.

– Этап Б (контракты и расширения): контракт `Analyzer` — `analyze(submission, context)` возвращает `AnalysisReport` с `analyzer_id` и `analyzer_version`; анализаторы `LengthAnalyzer` и `TemplateAnalyzer`; JSONL- и in-memory источники; один набор контрактных тестов; центральный сервис выбирает анализатор по реестру без ветвления по конкретным классам.

Траектория: Backend. Уровень: основной вариант и углубление Backend — зависимости собираются из конфигурации, некорректная конфигурация обнаруживается до первого запроса.

Входные данные: `data/submissions.jsonl` (12 учебных сдач из памятки) и `data/assignments.json` (задание `assignment-01` с разрешённым шаблоном `from typing import Iterable`).

## Программное и аппаратное обеспечение

| Компонент | Характеристики |
|---|---|
| Процессор | AMD Ryzen 5 7500F, 6 ядер / 12 потоков, 3,7 ГГц (Boost до 5,0 ГГц), L3 32 МБ, Zen 4 |
| Оперативная память | 32 ГБ DDR5-6400 (2 × 16 ГБ, ADATA XPG Lancer Blade) |
| Накопитель | Samsung 980 PRO 1 ТБ, NVMe PCIe 4.0 x4 |
| Операционная система | Windows 11 Pro 64-bit (10.0.26300) |
| Язык и окружение | Python 3.13.0 (CPython), виртуальное окружение venv |
| Библиотеки | pydantic 2.14.1 (strict DTO), setuptools ≥ 68 (сборка пакета) |
| Инструменты проверки | pytest 9.1.1, mypy 2.4.0 (strict), ruff 0.17.0 |
| Среда разработки | PyCharm 2025.2.1.1, git |

## Метод решения

Система разделена на слои с одной ответственностью у каждого. Внешние данные проверяются один раз на границе и дальше существуют только как неизменяемые доменные объекты. Конкретные реализации (репозиторий, источник, анализаторы) передаются в сервисы через конструктор, а сервисы зависят только от протоколов.

```
строка JSONL → источник (SourceRecord: номер строки + исходная строка)
   → parse_submission: строгий json.loads (без повторов ключей и NaN),
     проверка версии
   → SubmissionIn (Pydantic, strict, extra="forbid") → to_domain()
   → Submission (frozen dataclass)
   → SubmissionService.accept: SHA-256 канонического JSON
     → accepted / duplicate / conflict
   → InMemorySubmissionRepository

AnalysisService.run(submission_id, имя анализатора, контекст задания)
   → репозиторий.get → проверка задания → AnalyzerRegistry.create(имя)
   → analyzer.analyze → validate_report (проверка на границе) → AnalysisReport
```

Стабильный отпечаток сдачи — SHA-256 от канонического JSON (сортировка ключей, компактные разделители, UTF-8, запрет NaN). Порядок ключей не влияет на хеш, а лишний перевод строки в коде или другой `student_id` — влияет. На этом основано различие дубликата и конфликта.

## Описание программы

Пакет `school_lab` находится в `src/school_lab/`, тесты — в `tests/`, замеры — в `benchmarks/`. Назначение модулей:

| Модуль | Ответственность |
|---|---|
| `models.py` | Доменные модели: `Submission`, `AIDeclaration`, `Assignment`, `Evidence`, `Finding`, `AnalysisReport` (frozen dataclass, коллекции — кортежи) |
| `errors.py` | `SubmissionError` (code, field, message, `to_dict()`) и `AnalysisFailure` |
| `schemas.py` | Входные Pydantic-модели (DTO) со строгими типами, валидаторами и `to_domain()` |
| `parser.py` | Строка → `Submission`: строгий разбор JSON, проверка версии, перевод ошибок Pydantic в `SubmissionError` |
| `hashing.py` | `canonical_json`, `stable_hash`, `submission_hash` |
| `repository.py` | Протокол `SubmissionRepository` и in-memory реализация без перезаписи |
| `service.py` | `SubmissionService.accept` и адаптер `accept_payload` (сырой JSON → домен → приём) вместо HTTP |
| `sources.py` | `SourceRecord`, протокол `SubmissionSource`, `InMemorySource`, `JSONLSource` |
| `importer.py` | Импорт из любого источника с частичным успехом и номерами строк |
| `analyzers.py` | `AnalysisContext`, протокол `Analyzer`, `make_report`, `validate_report`, `LengthAnalyzer`, `TemplateAnalyzer` |
| `registry.py` | Реестр фабрик анализаторов `AnalyzerRegistry` |
| `analysis_service.py` | `AnalysisService.run` — запуск анализатора по имени и проверка отчёта |
| `app.py` | Точка сборки `build_application`, `AppConfig`, загрузка конфигурации и задания |

### *Валидация и ошибки*

Валидация выполняется на границе системы: в `parser.py` (формат JSON, версия схемы) и в `schemas.py` (Pydantic в strict-режиме с `extra="forbid"`). Аннотации типов не заменяют её: во время выполнения Python их не проверяет, а `bool` является подклассом `int`. Поэтому версия проверяется через `type(value) is int`, а `True`, строка `"1"` и отсутствие поля дают `schema_error`, а не `unsupported_version`.

| code | Когда возникает | Пример |
|---|---|---|
| `invalid_json` | Не JSON-объект: синтаксическая ошибка, пустая строка, массив, повтор ключа, NaN | `{broken` |
| `unsupported_version` | `schema_version` — целое число, но не 1 | `"schema_version": 2` |
| `schema_error` | Нет поля, лишнее поле, неверный тип, пустой id, время без часового пояса, `used=true` без `tool`/`purpose`, пустые одновременно `text` и `code` | `"student_id": "  "` |
| `conflict` | `submission_id` уже принят с другим содержимым | тот же `sub-001`, другой `text` |

Пользователь видит только `SubmissionError` с полями `code`, `field`, `message`; traceback предназначен разработчику. Ошибки анализа — отдельный тип `AnalysisFailure`. HTTP-статус (400 или 409) будет выбирать будущий HTTP-адаптер, доменные исключения от него не зависят.

### *Приём: дубликат и конфликт*

Дубликат — нормальная ситуация (повторная отправка той же версии), поэтому это результат `Acceptance(status="duplicate")`, и число сдач не растёт. Конфликт — отказ: под занятым id пришло другое содержимое, исходная запись не перезаписывается, сервис бросает `SubmissionError("conflict")`. Новая версия работы оформляется новым `submission_id`. Пример реального импорта четырёх строк после приёма учебного корпуса (успешный импорт 12 сдач дал `accepted = 12`, `errors = []`):

```
{"accepted": [], "duplicate": ["sub-001"],
 "errors": [
  {"line_no": 2, "code": "invalid_json", "field": "$", ...},
  {"line_no": 3, "code": "conflict", "field": "submission_id",
   "message": "submission_id already used with different content; ..."},
  {"line_no": 4, "code": "schema_error", "field": "student_id", ...}]}
```

### *Анализаторы, реестр и расширяемость*

`LengthAnalyzer` считает непустые строки `text` и `code` и возвращает `rubric_feedback` со `score=None`: объём работы не оценивает её корректность. `TemplateAnalyzer` ищет точные блоки строк разрешённого

шаблона и возвращает `similarity` с явным пояснением, что совпадение разрешено; при контексте другого задания он отказывает. Номера строк evidence относятся к исходному тексту, а не к тексту после удаления пустых строк.

Протокол `Analyzer` описывает только форму (id, версия, `analyze`). Реализации от него не наследуются: класс подходит, если у него есть нужные члены (структурная типизация). Наследование задаёт иерархию и общее поведение, протокол — лишь требования к интерфейсу. Поведение протокол не гарантирует, поэтому результат стороннего анализатора проверяется функцией `validate_report` (связь с версией сдачи и конфигурацией, `score` — конечное число 0–1 или `None`, не `bool`; диапазоны evidence), а обещания реализаций — контрактными тестами.

`AnalysisService` получает анализатор по имени из `AnalyzerRegistry` и не содержит ветвления по классам. Третий анализатор (например, проверка AI-декларации) подключается одной строкой регистрации без изменения сервиса — это проверено тестом `test_third_analyzer_plugs_in_by_registration_only`.

### *Источники и управление ресурсом*

Оба источника реализуют `open()` — контекстный менеджер, выдающий итератор записей. `InMemorySource` хранит копию списка и ресурсом не владеет. `JSONLSource` читает файл построчно и владеет файлом: файл открывается в `open()` и закрывается в `finally`, поэтому он закрывается и при полном чтении, и при раннем выходе (`break`, `next` и выход из `with`), и при исключении потребителя; само исключение не подавляется. Импорт только потребляет записи, ловит лишь `SubmissionError` и пишет её в отчёт с номером строки; неожиданный дефект пробрасывается дальше. Отсутствие файла — ошибка источника (`OSError`), а не ошибка отдельной сдачи.

### *Сборка приложения*

`build_application(config)` — единственное место, где выбираются конкретные классы. Каждый вызов создаёт независимые репозиторий, реестр и сервисы, глобального изменяемого состояния нет. `AppConfig` строго разбирается из JSON; неизвестный, повторяющийся или пустой список анализаторов, а также фабрика с чужим `analyzer_id` приводят к `AnalysisFailure` при сборке — до первого пользовательского запроса.

## Выполненные требования

| Требование | Реализация | Проверка |
|---|---|---|
| Пакет в src-layout, установка в чистое окружение | `pyproject.toml`, `src/school_lab` | `pip install ".[dev]"` без `-e` в новом venv, все тесты проходят |
| Модель `Submission`, неизменность | `models.py` (frozen) | `test_models.py`, `test_*_does_not_change_submission` |
| Разбор JSON, обязательные поля, часовой пояс, AI-декларация | `parser.py`, `schemas.py` | `test_parser.py` (22 теста) |
| Ошибка `code`/`field`/`message`, три вида ошибок | `errors.py`, `parser.py` | `test_parser.py` |
| Дубликат не увеличивает число сдач, конфликт не перезаписывает | `service.py`, `repository.py` | `test_service.py` |
| Битая строка не останавливает импорт, номер строки в отчёте | `importer.py` | `test_importer.py::test_mixed_lines` |
| Входные данные не изменяются | `parser.py` | `test_input_is_not_mutated` |
| Pydantic DTO → доменный объект, тестовый адаптер вместо HTTP | `schemas.py`, `accept_payload` | `test_service.py` |
| Контракт `Analyzer`, отчёт с `analyzer_id` и версией | `analyzers.py` | `test_analyzers.py` |
| `LengthAnalyzer` и `TemplateAnalyzer` | `analyzers.py` | `test_analyzers.py`, `test_analyzer_contract.py` |
| Диапазоны evidence, проверка отчёта на границе | `validate_report` | `test_validate_report.py` (33 теста) |
| Ошибка анализатора | `AnalysisService`, `validate_report` | `test_run_rejects_report_*` |
| Реестр, сервис без ветвления по классам | `registry.py`, `analysis_service.py` | `test_registry.py`, `test_third_analyzer_*` |
| JSONL- и in-memory источники, общие контрактные тесты | `sources.py` | `test_sources.py`, `test_analyzer_contract.py` |
| Освобождение ресурса при ошибке и раннем выходе | `JSONLSource.open` (`finally`) | `test_jsonl_file_closed_*`, `test_import_closes_file_*` |
| Конфигурация собирает зависимости, нет глобального состояния, подмена источника в тесте | `app.py` | `test_app.py` |
| Углубление: некорректная конфигурация до первого запроса | `build_application`, `AppConfig` | `test_bad_config_*`, `test_factory_with_wrong_id_*` |

## Команды проверки

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
python -m pytest                    # 182 passed
mypy src tests benchmarks           # Success: no issues found in 28 source files
ruff check .                        # All checks passed!
python benchmarks/run_benchmarks.py # замеры → docs/benchmark_results.json
```

## Тестирование

Всего 182 теста; используются fixtures (`conftest.py`), `pytest.mark.parametrize` и fixtures с `params` для контрактных тестов, которые запускают один набор проверок для каждой реализации протокола.

| Файл тестов | Число | Что проверяется |
|---|---|---|
| `test_parser.py` | 22 | корректная запись; битый JSON, массив, повтор ключа, NaN; версия; пустые поля; часовой пояс; неизвестное поле; типы; AI-декларация; неизменность входа |

| | | |
|---|---|---|
| `test_hashing.py` | 5 | устойчивость хеша, влияние кода и `student_id`, независимость от порядка ключей |
| `test_models.py` | 3 | неизменяемость моделей, `stable_hash` |
| `test_service.py` | 9 | репозиторий; accepted / duplicate / conflict; новая версия; адаптер |
| `test_importer.py` | 8 | частичный успех, номера строк, учебный корпус, закрытие файла, проброс неожиданной ошибки |
| `test_sources.py` | 13 | оба источника; закрытие файла; контракт источников |
| `test_analyzers.py` | 27 | `nonempty_line_numbers`, `make_report`, `template_ranges`, оба анализатора |
| `test_validate_report.py` | 33 | проверка отчёта: версия, конфигурация, id, время, `kind`, `score`, evidence |
| `test_analyzer_contract.py` | 30 | контракт анализаторов на всех 12 учебных сдачах |
| `test_registry.py` | 7 | регистрация, создание, отказы |
| `test_analysis_service.py` | 10 | запуск, отказы, ошибка анализатора, третий анализатор |
| `test_app.py` | 15 | сборка, конфигурация, сквозной сценарий |

## Тест производительности

Методика. Скрипт `benchmarks/run_benchmarks.py` генерирует корпус из учебных сдач с уникальными `submission_id` (1 000, 10 000 и 50 000 строк; 0,5, 5 и 25 МБ). Время измеряется `time.perf_counter` как медиана нескольких запусков (источники — 5, импорт — 3, анализ — 50), пиковая память — `tracemalloc` (учитываются только выделения Python). Все замеры однопоточные, на оборудовании из раздела выше.

*Таблица 1. Чтение источника: время и пиковая память*

| Сдач | Файл | JSONL, мс | In-memory, мс | JSONL, пик | In-memory, пик |
|---|---|---|---|---|---|
| 1 000 | 0,5 МБ | 3,1 | 3,3 | 54 КБ | 1,0 МБ |
| 10 000 | 4,9 МБ | 23,9 | 29,8 | 54 КБ | 9,7 МБ |
| 50 000 | 24,7 МБ | 123,8 | 149,6 | 53 КБ | 48,5 МБ |

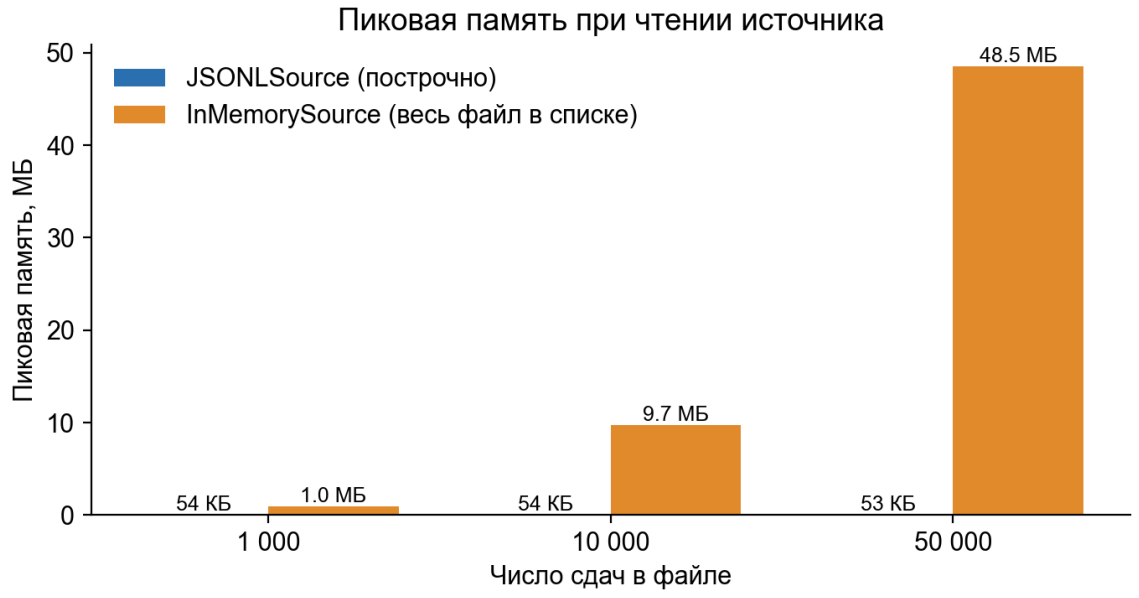



*Рисунок 1. Пиковая память при чтении источника*

`JSONLSource` держит в памяти одну строку, поэтому его пиковая память постоянна (≈ 54 КБ) и не зависит от размера файла. In-memory источник, загруженный из того же файла, хранит все строки и растёт линейно: 50 000 сдач — около 48,5 МБ, почти в 1 000 раз больше. По времени источники близки (JSONL быстрее на величину до 20 %, так как не строит промежуточный список); различие — именно в памяти. Ленивость генератора не гарантирует малый расход памяти всей программы: репозиторий всё равно хранит принятые сдачи.

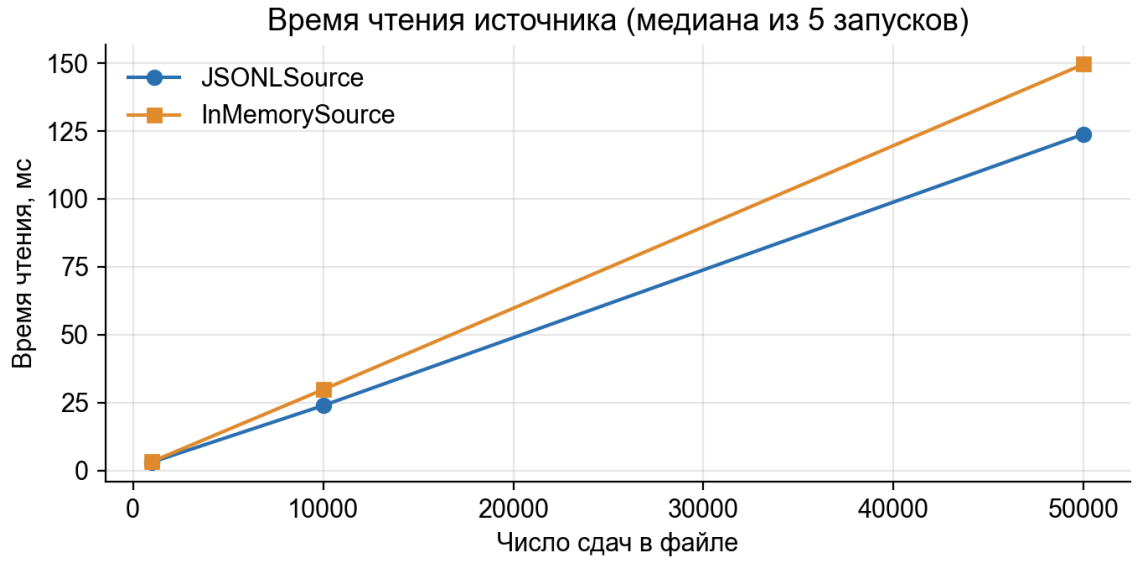



*Рисунок 2. Время чтения источника*

*Таблица 2. Полный импорт (разбор, валидация, хеш, репозиторий)*

| Сдач | Импорт, мс | Из них разбор, мс | На сдачу, мкс | Сдач в секунду |
|---|---|---|---|---|
| 1 000 | 36,6 | 12,8 | 36,6 | 27 296 |
| 10 000 | 356,9 | 141,0 | 35,7 | 28 020 |

Импорт линеен: около 36 мкс на сдачу (≈ 28 тыс. сдач в секунду). Строгий разбор JSON и Pydantic-валидация занимают около 40 % времени; остальное — канонический JSON, SHA-256 и работа репозитория (хеш считается дважды при проверке дубликата).

*Таблица 3. Анализ одной сдачи и цена проверки отчёта на границе*

| Анализатор | Вход | Анализ, мкс | + проверка до, мкс | + проверка после, мкс |
|---|---|---|---|---|
| length | 12 учебных | 30,0 | 61,9 | 53,3 |
| length | 2000 строк | 5 352,0 | 103 879,7 | 5 675,5 |
| template | 12 учебных | 26,8 | 42,2 | 48,3 |
| template | 2000 строк | 363,4 | 383,7 | 421,7 |

Измерение выявило дефект. На сдаче из 2 000 строк проверка отчёта `LengthAnalyzer` занимала 103,9 мс при самом анализе около 5,4 мс:

`validate_report` заново разбивал поле на строки для каждой evidence, то есть работал за O(n²). После исправления (число строк считается один раз на поле) время стало 5,7 мс — в 18 раз быстрее, накладные расходы проверки упали с 1 899 % до 6 %. На маленьких учебных сдачах проверка добавляет десятки микросекунд, что несущественно.

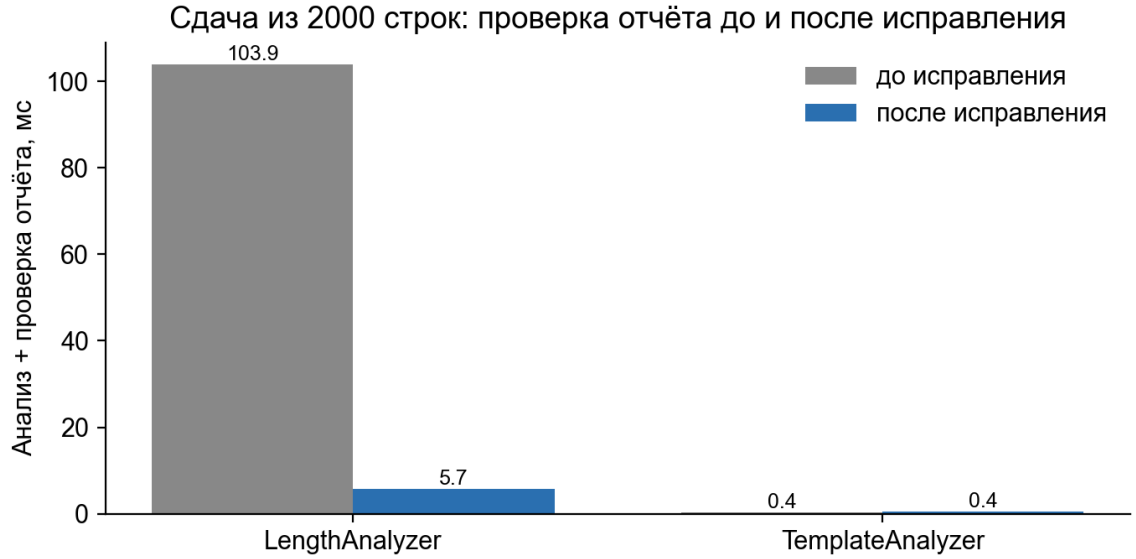



*Рисунок 3. Проверка отчёта на сдаче из 2 000 строк*

Эффект углубления. Сборка приложения с проверкой конфигурации занимает около 1,8 мкс и выполняется один раз. Взамен ошибка вида «неизвестный анализатор в конфигурации» обнаруживается при запуске, а не при первом пользовательском запросе, когда часть данных уже могла быть принята.

## Известные ограничения

– Репозиторий хранится в памяти: после завершения процесса принятые сдачи исчезают (постоянное хранение — ЛР 3).

– Последовательность `get` → `add` не атомарна: при параллельной работе два одинаковых id могут пройти проверку одновременно; для синхронной ЛР 1 это допустимо.

– Импорт атомарен на уровне одной строки, а не файла: строки, принятые до ошибки, остаются в репозитории.

– Принятый код не исполняется и не проверяется на синтаксис — это данные, а не программа.

– В отчёт об ошибке схемы попадает только первая ошибка Pydantic в строке.

– `TemplateAnalyzer` ищет только точное совпадение строк; нечёткое сходство, переименования и AST не учитываются. Совпадение не доказывает и не опровергает плагиат.

– `validate_report` проверяет результат, но не защищает от вредоносного плагина: анализатор выполняется с правами процесса.
– Размер одной строки JSONL не ограничивается; замеры памяти `tracemalloc` учитывают только выделения Python и сделаны на одной машине.

## Использование ИИ

Использовался Claude (Claude Code). В этапе А ИИ выдавал заготовки модулей с TODO-комментариями и тесты, код в TODO писал автор. В этапе Б задачи описывались словами: модели отчёта, `LengthAnalyzer`, `template_ranges`, реестр, `AnalysisService`, `InMemorySource` и `JSONLSource` написаны автором с подсказками; `TemplateAnalyzer`, `validate_report`, `import_source`, контрактные тесты, `build_application`, скрипт замеров, часть тестов и черновик отчёта — ИИ по просьбе автора. Весь код от ИИ прочитан и проверен тестами, mypy и ruff. Подробный перечень и найденные при разборе ошибки — в `AI_USAGE.md`.

## Вывод

Реализована основа системы приёма лабораторных: устанавливаемый пакет со строгой валидацией на границе, неизменяемыми доменными моделями, устойчивым SHA-256 отпечатком сдачи и различением дубликата и конфликта. Анализаторы подключаются через протокол и реестр фабрик, поэтому новый анализатор добавляется регистрацией без изменения центрального сервиса, а результат любого анализатора проверяется на границе.

Работа показала, что аннотации и протоколы описывают форму, но не поведение: инварианты обеспечиваются явной валидацией, а обещания реализаций — контрактными тестами (182 теста, mypy strict и ruff без замечаний). Источник, владеющий файлом, гарантирует его закрытие при ошибке и раннем выходе. Измерения подтвердили, что построчное чтение держит память постоянной, и выявили квадратичную проверку отчёта, исправление которой ускорило её на больших сдачах в 18 раз.